

Facilitator answer key — Holocron capstone worked solution. Facilitator-only — lives in the facilitator folder of every repo; do not hand to participants. Part of the internally-consistent set in this folder (see `README.md`).

TCD-light — Holocron (release-1 slice)

Product: **Holocron** — centralized, governed management of UI strings. Scale target: 150+ countries, 20–30+ consuming apps, 30–40 languages, `en-US` source of truth, ~500,000 strings, Internal data classification. This TCD covers the **release-1 slice**: access roles, product setup, create/edit/publish strings, version history + audit, search, delivery to consumers, review request/complete, translation variants. Out of scope: aliases, advanced reviewer config, export/import, rollback, AI translation, Figma, screenshots.

1. Architecture stance

A **modular service on AKS** — one deployable with internally separated modules, not a distributed microservice mesh and not a monolith we can't scale. The decisive move is **two physically separate paths sharing one PostgreSQL system of record**: a low-traffic **management/authoring path** (create/edit/publish/review, management app at 99.5%) and a high-traffic, **read-optimized delivery path** fronted by **Azure Cache for Redis** and served through **API Management + Application Gateway** (delivery at 99.9%, p95 ≤ 200ms). Business case: 20–30+ apps fetching strings on every render generate orders of magnitude more traffic than the handful of content editors — collapsing both onto the authoring path would let read spikes starve publishing, and would force us to scale the expensive authoring/audit machinery just to serve cache-friendly reads. Operationally, the split lets us scale, deploy, and set SLOs for delivery independently: delivery pods scale horizontally on read QPS and can survive a management-app deploy or outage

(they serve from Redis + Postgres read replica), while authoring changes never risk the delivery contract. Starting modular (not micro) keeps the release-1 team small and the transaction boundary — publish + audit-write — inside one process; we extract a service only when a module's scale or ownership demands it.

2. Integration map

System	Owner	Sync / Async	R / W	Failure handling
Azure Database for PostgreSQL (system of record)	Platform	Sync	R/W	Primary write target; Failover to replica if unreachable (serv
Azure Cache for Redis (delivery read cache)	Platform	Sync (read) / Async (fill)	R (delivery) / W (on publish)	Cache miss or evi backfill. Redis dov reads (higher later
Microsoft Entra ID (identity / SSO + role scoping)	Security	Sync	R	Every manageme Token-endpoint or tokens; new sign-i
Application Gateway + Azure API Management (delivery API)	Platform	Sync	R	Terminates TLS, e burst 100) and tok unhealthy → 503 strings.
Azure Event Hubs (audit / event stream)	Platform	Async	W	Every mutating op buffer + retry with commits to Postgr blocks the write).
Blob Storage (audit archive)	Platform	Async	W	Consumes Event per Internal policy window covers rep
Azure Key Vault (secrets)	Security	Sync (startup) / cached	R	Connection string: unreachable → ru rotation, alert.
Azure Monitor (observability)	Platform	Async	W	Metrics/logs/trace: is non-blocking; g;
Consuming apps (20–30+)	Product teams	Sync	R	Call GET /v1/products/{locale=... with a side and tolerate s

3. Threat-model summary (STRIDE)

Threat (STRIDE)	On the flow	Mitigation (Azure)
S poofing a consumer	Forged/borrowed token hits the delivery API	Entra ID token validation at API Management edge; per-app identities with scoped audiences; no anonymous delivery.
T ampering with published content	Altering strings in transit or at rest between publish and delivery	TLS end to end; Postgres is the only write path (delivery is read-only); Redis fill is derived from committed rows; RBAC-gated publish.
R epudiation of a publish/approve	Editor/reviewer denies making a change	Immutable <code>AuditEntry</code> per mutating op → Event Hubs → Blob archive; actor identity from Entra ID token; publish-time review override is itself audited.
E levation of privilege via role scope	A viewer/editor performs a publish or review they shouldn't	Entra ID role scoping enforced server-side per operation (RBAC), not in the client; product-scoped roles; least-privilege delivery identity.
D enial of service on delivery	One consumer floods the fetch endpoint	Per-consumer rate limit 50 req/s / burst 100 at API Management; Redis absorbs read load; delivery path scales independently and isolates authoring.

4. SLOs (3)

SLO	Target	Defense (user behavior) + verification
Delivery fetch latency	$p95 \leq 200\text{ms}$ / $p99 \leq 500\text{ms}$	Apps fetch strings on render — latency is on the critical UI path, so it must beat a human-noticeable frame. Redis serves the hot path. Verify via Azure Monitor percentile dashboards + alerts on the delivery API.
Delivery availability	99.9%	Consuming apps depend on delivery for every localized screen across 150+ countries; the read path is deliberately isolated so authoring incidents can't take it down. Verify via synthetic probes + edge success-rate monitors.
Publish → delivery propagation	$\leq 60\text{s}$ normal / < 5s for critical/legal - flagged strings	Editors expect a published change to reach apps quickly but not instantly; 60s lets us cache aggressively while staying operationally honest. Compliance-critical strings carry a critical/legal flag that forces synchronous cache invalidation (< 5s) — the outcome of the Legal/Compliance negotiation (SEP §3). Verify with a publish-to-fetch timing probe in Azure Monitor, split by flag.

5. Top 3 trade-offs

Option A	Option B	Choice	Accepted cost
Redis read-cache in front of delivery	Strong-consistency reads straight from Postgres	A — cache	Up to ≤60s staleness for normal strings; critical/legal flagged strings get synchronous <5s invalidation (SEP §3)
Reviews optional by default + audited publish-time override	Mandatory review gates on every publish	A — optional + override, flagged exception	A normal string can ship unreviewed (audit, not prevention); critical/legal flagged strings require enforced pre-publish compliance review (SEP §3)
Immutable namespaced keys	Mutable/renamable keys	A — immutable	Renaming means create-new + deprecate-old, more editor friction

6. Stakeholder sign-off

Stakeholder	Interest	Status
Product lead (Holocron)	Scope slice + release-1 value	Approved
Platform / SRE lead	AKS topology, delivery SLOs, Redis/Postgres split	Approved
Security / Identity lead	Entra ID scoping, RBAC, audit immutability	Approved (pending Key Vault rotation runbook)
Consuming-app representative	Delivery API contract + ≤ 60 s propagation tolerance	Approved
Legal / Compliance (governance owner)	Pre-publish review for compliance- critical strings + audit trail	Approved — critical/legal flag enforces pre-publish review + < 5 s propagation (SEP §3)